Chapter 6

Data Types

Topics

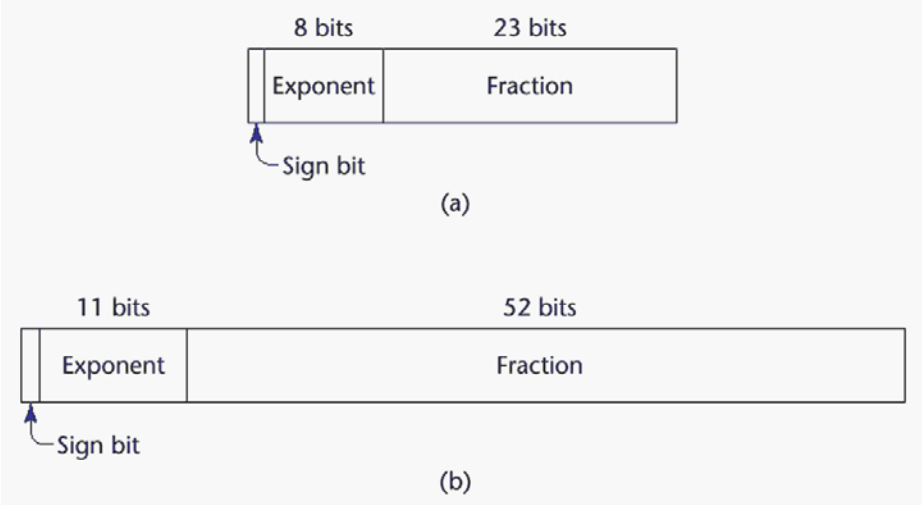
- Introduction
- Primitive data types
- User-Defined Ordinal Types
- Structured data types
  - Array Types
  - Record Types
  - Union Types
  - Lists/tuples/sets, Pointer and Reference Types
- Type checking

Introduction

- A *data type* defines a collection of data values and a set of predefined operations on those values
- One design issue for all data types
  - What operations are defined and how are they specified?
- How well the data types match the entities in the real-world problem space?

Primitive data types

- not defined in terms of other data types
- Integer
- Floating point
- Complex numbers
- Decimal
- Boolean
- Characters
- Strings



User-defined ordinal types

- Enumeration Types
  - C# example

```
enum days {mon, tue, wed, thu, fri, sat, sun};
```

- Subrange Types
  - Ada's design

```
type Days is (mon, tue, wed, thu, fri, sat, sun);
subtype Weekdays is Days range mon..fri;
subtype Index is Integer range 1..100;
```

Array Types

Arrays

- An array is an aggregate of homogeneous data elements in which an individual element is identified by its position in the aggregate, relative to the first element.

Heterogeneous Arrays

- A *heterogeneous array* is one in which the elements need not be of the same type
- Supported by Perl, Python, JavaScript, and Ruby

Associative Arrays

- An *associative array* is an unordered collection of data elements that are indexed by an equal number of values called *keys*
  - User defined keys must be stored

- Associative Arrays in Perl
  - Names begin with `%;` literals are delimited by parentheses

```
%hi_temps = ("Mon" => 77, "Tue" => 79, “Wed” => 65, …);
```

  - Subscripting is done using braces and keys

```
$hi_temps{"Wed"} = 83;
```

  - Elements can be removed with `delete`

```
delete $hi_temps{"Tue"};
```

Array Design Issues

- What types are legal for subscripts?
- Are subscripting expressions in element references range checked?
- When are subscript ranges bound?
- When does allocation take place?
- Are ragged or rectangular multidimensional arrays allowed, or both?
- What is the maximum number of subscripts?
- Can array objects be initialized?
- Are any kind of slices supported?

Arrays

- Five Categories of Arrays
  (based on **subscript value range binding and storage binding**)
  - Static
  - Fixed stack dynamic
  - Stack dynamic
  - Fixed Heap dynamic
  - Heap dynamic

Types of Array

- *Static*:
  - **i. range of subscripts: statically bound**
  - **ii. storage bindings: statically bound**
  - – Advantage: efficiency (no dynamic allocation)

```
// C++ example
int x[4];   // global array

void foo(void) {
   ...
   static int y[4];  // static local array
}

int main(void){
   ...
}
```

Types of Array

- *Fixed stack-dynamic*:
  i. **range of subscripts: statically bound**
  ii. **storage is bound: at elaboration time**
  iii. Advantage: space efficiency

```
// C++ example

void foo(void) {
   int y[4];   // fixed stack-dynamic array
   ...
}
```

Types of Array

- *Stack-dynamic*:
  i. Subscript ranges: dynamic and fixed
  ii. Storage allocation: dynamic (done @ runtime) fixed
  – Advantage: flexibility (the size of an array need not be known until the array is to be used)

Ada has stack dynamic arrays:

```
Get(List_len)
declare
   List: array(1..List_Len) of Integer;
   begin
   ...
   end;
```

Types of Array

- *Fixed heap-dynamic*:
  - Binding of subscript ranges: dynamic and fixed
  - Binding of storage: dynamic and fixed
  - storage is allocated from heap, not stack
- In Java, all arrays are fixed heap-dynamic arrays.

Types of Array

- *Heap-dynamic*:
  - subscript range: dynamic and NOT fixed
  - storage bindings: dynamic and NOT fixed
  - Advantage: flexibility (arrays can grow or shrink during program execution)

- C++'s and Java's Vector and ArrayList are all examples of heap dynamic arrays.

```
ArrayList<Integer> intList = new Arraylist();
intList.add(nextOne);
```

Record Types

Records in C and C++

- Are supported with struct.
- The size of a record must be known at compile time.

```
// This is OK.
struct foo1 {
   int x;
   int y[3];
};

// This is not OK.
struct foo2 {
   int x;
   int y[x];
};

// This is OK.
struct foo3 {
   int x;
   int *y; // pointer to dynamic array
};
```

Unions Types

Unions Types

- A *union* is a type whose variables are allowed to store different value types at different times during program execution.

```
union symbol {
    int num;
    char letter;
    bool trueOrFalse;
};

symbol x;
```

- x can hold either an integer, character, or bool, but not all three at the same time.
- To set the value of a union, we can use dot notation: e.g. x.num = 100;
- Internally, just enough space is allocated for the union so as to accommodate the largest possible data value

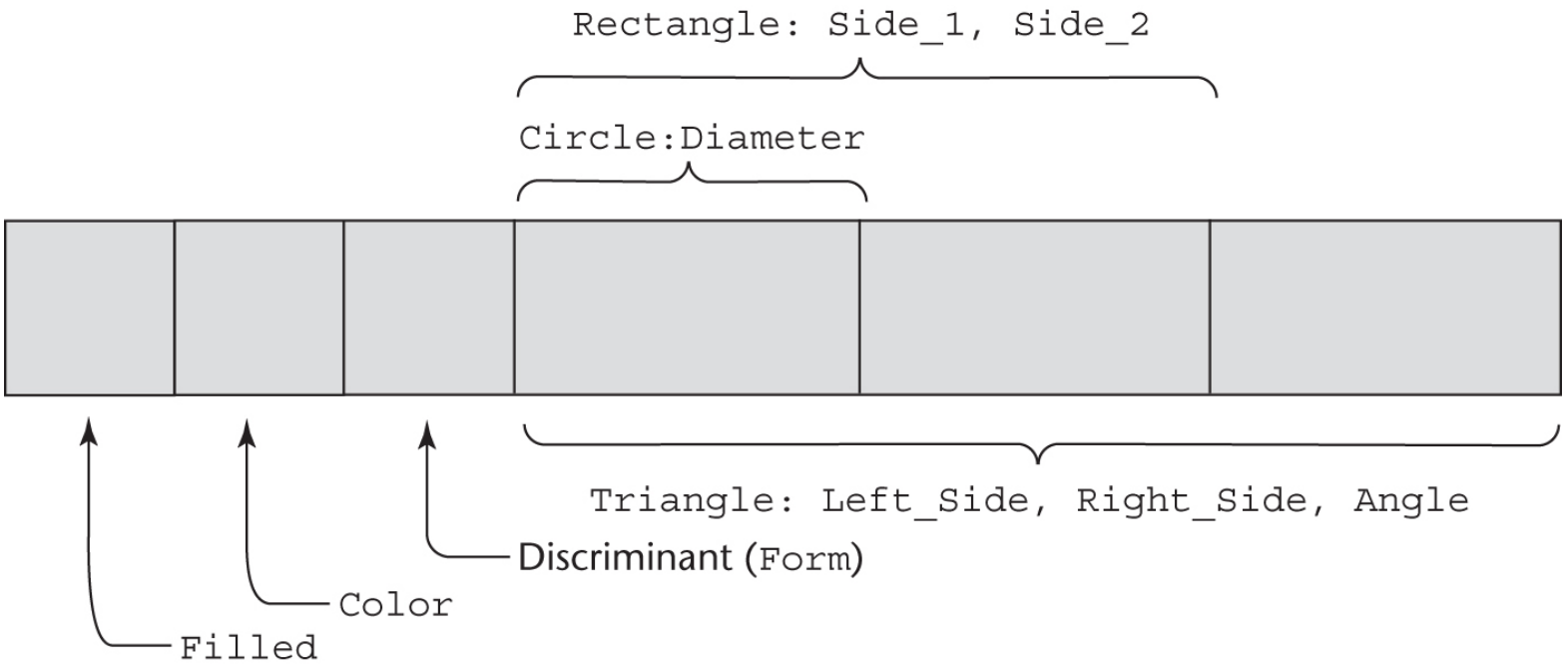
Discriminated vs. Free Unions

- Fortran, C, and C++ provide union constructs in which there is no language support for type checking; the union in these languages is called *free union*
- Type checking of unions require that each union include a type indicator called a *discriminant*
  - Supported by Ada

Ada Union Types

```
type Shape is (Circle, Triangle, Rectangle);
type Colors is (Red, Green, Blue);
type Figure (Form: Shape) is record
   Filled: Boolean;
   Color: Colors;
   case Form is
       when Circle => Diameter: Float;
       when Triangle =>
             Leftside, Rightside: Integer;
             Angle: Float;
       when Rectangle => Side1, Side2: Integer;
   end case;
end record;
             Figure_2 : Figure(Form => Triangle);
```

Ada Union Type Illustrated



A discriminated union of three shape variables

Union in C++

- Suppose you need to create a data type to store a person's contact information. If the person is an employee, store his/her phone extension (as a four digit integer). If the person is not an employee, store his/her full phone number (as a C-style string).

```
struct phone_entry {
    string name;
    boo is_employee;
    union {
        int extension;
        char phone_no[10];
    }
};
```

Lists, tuples, and sets

- Lists are mutable and ordered collection, enclosed in brackets:
  - l = [1, 2, "a"]
- Tuples are immutable and ordered collection, enclosed in parentheses:
  - t = (1, 2, "a")
- A set is an unordered collection with no duplicate elements.
  - s = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}

Strong typing vs. weak typing

TYPE CHECKING

Type Checking

- *Type checking* is the activity of ensuring that the operands of an operator are of compatible types
- A *compatible type* is one that is either legal for the operator, or is allowed under language rules to be implicitly converted, by compiler- generated code, to a legal type
  - This automatic conversion is called a *coercion*.
- A *type error* is the application of an operator to an operand of an inappropriate type

Type Checking (continued)

- If all type bindings are static,
  - nearly all type checking can be static
- If type bindings are dynamic,
  - type checking must be dynamic
- A programming language is *strongly typed* if type errors are always detected

Strong Typing

Language examples:

- C and C++ are not: parameter type checking can be avoided; unions are not type checked
- Ada is, almost (`UNCHECKED CONVERSION` is loophole)

  (Java and C# are similar to Ada)

Type Conversions

- Narrowing conversion
  - converts the value of a type to another type that cannot store all the values of the original type
    - e.g., convert double to float
- Widening conversion
  - converts the value to a type that include at least approximations to all of the values of the original type
    - e.g., convert integers to double

Implicit type conversion

- A *coercion* is an implicit type conversion
- is useful in mixed-mode expressions
  - expressions containing variables of different types.

Explicit Type Conversions

- Called *casting* in C-based language
- Examples
  - `Java: (int)angle`
  - `Ada: Float(sum)`

- Most languages allow widening conversions to be implicit. C and C++, however, allow narrowing conversions to be implicit as well.

```
int x;
float y;
...
x = y;
```

Strong typing and weak typing

- "strong typing" implies that the programming language places severe restrictions on the intermixing that is permitted to occur.
- weakly typed programming languages are those that support either high degree of implicit type conversion (nearly all languages support at least one implicit type conversion), ad-hoc polymorphism (also known as *overloading*) or both.

	Weak Typing	Strong Typing

Pseudocode	`a = 2`
	`b = '2'`
	
	`concatenate(a, b) # Returns '22'`
	`add(a, b) # Returns 4`
		`a = 2`
		`b = '2'`
		
		`#concatenate(a, b) # Type Error`
		`#add(a, b) # Type Error`
		`concatenate(str(a), b) # Returns '22'`
		`add(a, int(b)) # Returns 4`